

Game Programming using Unity

Lab Assessment - Challenging Task

Project: **Football Game (Player vs AI Opponent)** built in Unity 2022.3.9f1

Student Name	Md Shohel Hossain
Registration No.	24BCG10042
Course / Section	BL2026270100720 C11 + C12
Unity Version	2022.3.9f1 (Built-in Render Pipeline)
Submission Date	06 October 2026

All Unity screenshots in this report are taken from my own project (rebuild football - SampleScene). Scripts quoted are from Assets/main script/. Items marked **Pending item** are requirements that are not yet present in the project and must be completed and screenshotted before the final submission.

Project Overview

The project is a 3D football (soccer) game. The player controls a rigged Mixamo character with the keyboard, picks up and kicks a ball with physics, and competes against an AI opponent that chases the ball and shoots at goal. A UI text shows the score and the players/ball are reset after every goal.

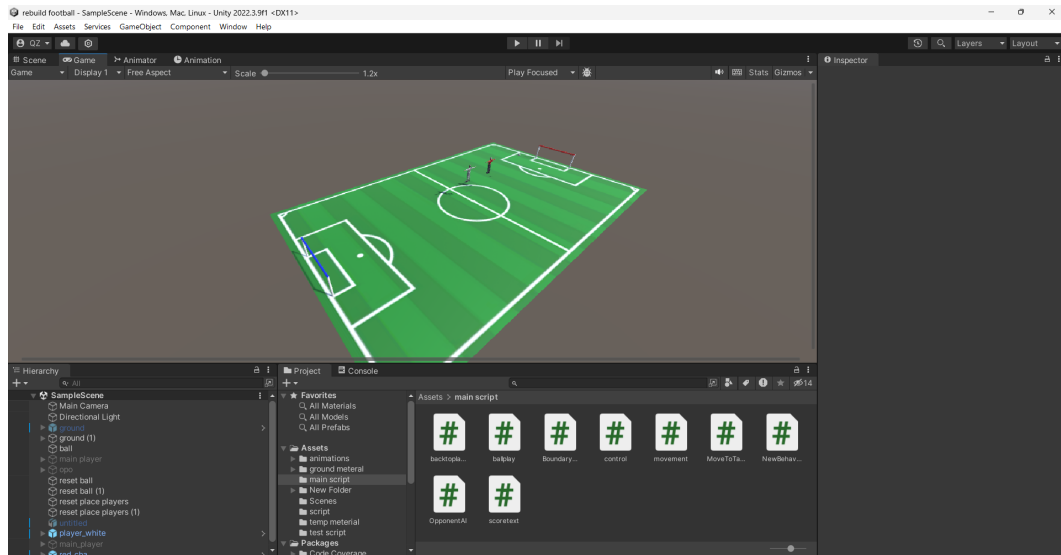


Fig. 0 – Unity Editor (Game view): football pitch, two characters, Hierarchy (Main Camera, Directional Light, ground, ball, main player, opo, reset points) and Project window showing the main script folder.

Project folders

Folder	Contents
Assets/New Folder/characters	Mixamo FBX characters (Ch08, Ch38, Ch42), animation FBX/anim files, character materials and textures
Assets/animations	Animator Controllers (player_ani, opo_ani, player_av_ani), character prefabs (player_white, red_cha, black_cha), animation clips
Assets/ground meteral	Pitch textures (jpg), pitch materials, ground prefab, audience model/textures
Assets/main script	backtoplace, ballplay, BoundaryController, control, movement, MoveToTarget, OpponentAI, scoretext
Assets/Scenes	SampleScene.unity

Q1. Asset Integration, Materials, Shaders and Lighting

Implementation

- **Imported 3D models:** two humanoid FBX characters (Mixamo, Ch38_nonPBR__player.fbx for the player and Ch42_nonPBR--oponent.fbx for the opponent), a football (ball package), goal posts, and an audience stand model (untitled.blend).
- **Imported textures:** pitch textures such as soccer-field_95169-1934.jpg, top-view-football-field...jpg, gro 1.jpg, audience textures, and character Diffuse / Normal / Glossiness maps (Ch08_1001_Diffuse.png, ..._Normal.png).
- **Materials:** one material per texture in Assets/ground meteral/Materials (e.g. soccer-field_95169-1934.mat) applied to the **floor** plane, and character materials Ch38_body, Ch42_Body, Ch42_Hair in caractor/materials. The pitch markings are visible because the white-line texture is assigned as the Albedo map of the floor material.
- **Lighting:** a Directional Light (sun) with soft real-time shadows; the floor has Cast Shadows = On and Receive Shadows ticked so the players cast shadows on the pitch (see Fig. 1.2).



Fig. 1.1 – Scene view with the **floor** object selected: Mesh Renderer, Materials, Lighting (Cast/Receive Shadows) and Probes settings in the Inspector.

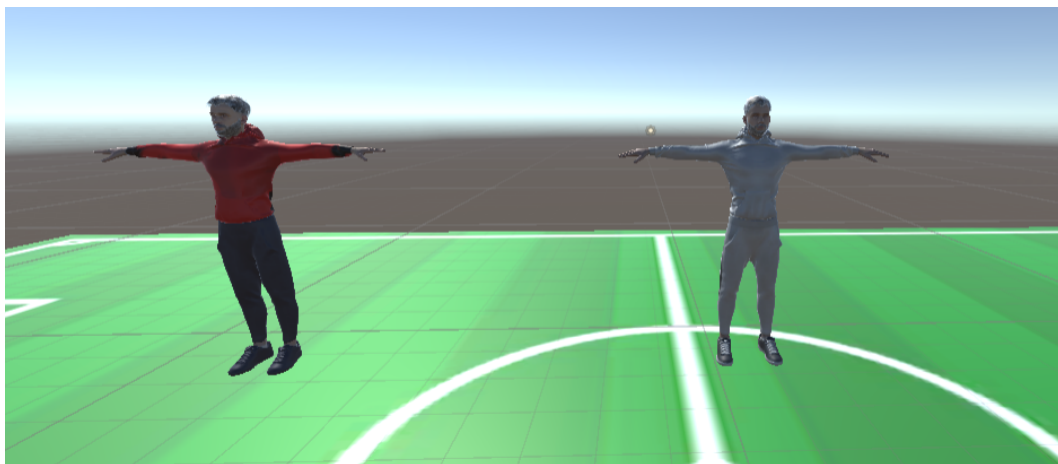


Fig. 1.2 – Imported Mixamo characters with their textured materials on the pitch (red kit = opponent, grey = player).

Explanation - how texture, material, shader and lighting work together

A **texture** is an image (diffuse/normal/gloss) that is mapped onto the model using its UV coordinates. A **material** stores which textures and values (colour, tiling, smoothness) an object uses. The **shader** assigned to that material is the GPU program that combines the textures and material values with the scene lights to calculate every pixel. The **lighting** (Directional Light, shadows, ambient) supplies the light the shader uses. In my scene the Standard shader combines the pitch texture with the sun direction, producing the lighter and darker bands across the grass

and the soft shadows under the players; removing the light or switching to an Unlit shader makes the pitch look flat.

Pending item: Shader comparison and second lighting setup. Select the floor material → change Shader from Standard to Unlit/Texture and capture both Inspector + Game view. Then create a second lighting setup (e.g. night: Directional Light intensity 0.2 + blue tint, plus a Point Light) and capture it. Paste the screenshots here and write your own observation.

Q2. Terrain, Character, Animation and Audio Integration

Rigged and animated characters

Both characters are Mixamo humanoid rigs (bone names mixamorig7:Hips, LeftUpLeg, Spine ... are visible in Fig. 2.2). Several animations were imported and organised in an Animator Controller:

Animation clip	Used for	Controller
breating idle	Idle (default state)	player_ani
Strike Foward Jog / Fast Run	Running (toRun)	player_ani / opo_ani
Dribble	Running with the ball	player_ani
kick	Shooting (tokick)	player_ani
Soccer Tackle	Slide tackle (toslide)	player_ani
Soccer Spin	Spin skill (spinSkill, Left Shift)	player_ani

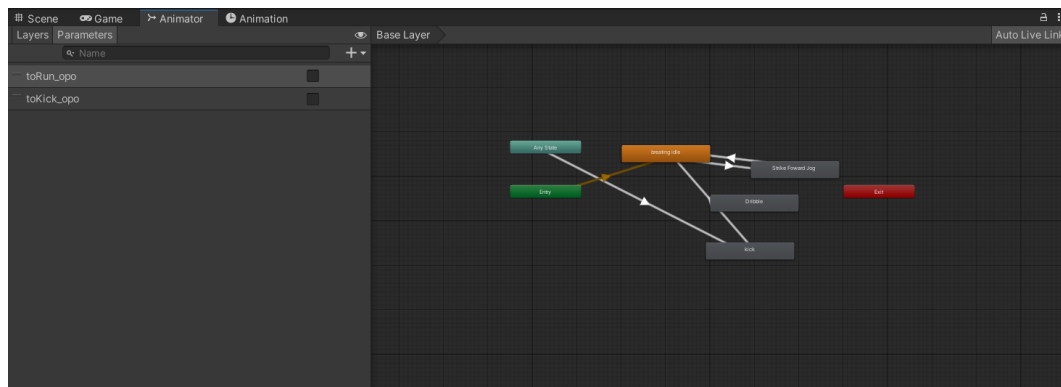


Fig. 2.1 – Animator window of the opponent controller: Entry → breating idle (default) with states Strike Foward Jog, Dribble and kick; parameters toRun_opo and toKick_opo.

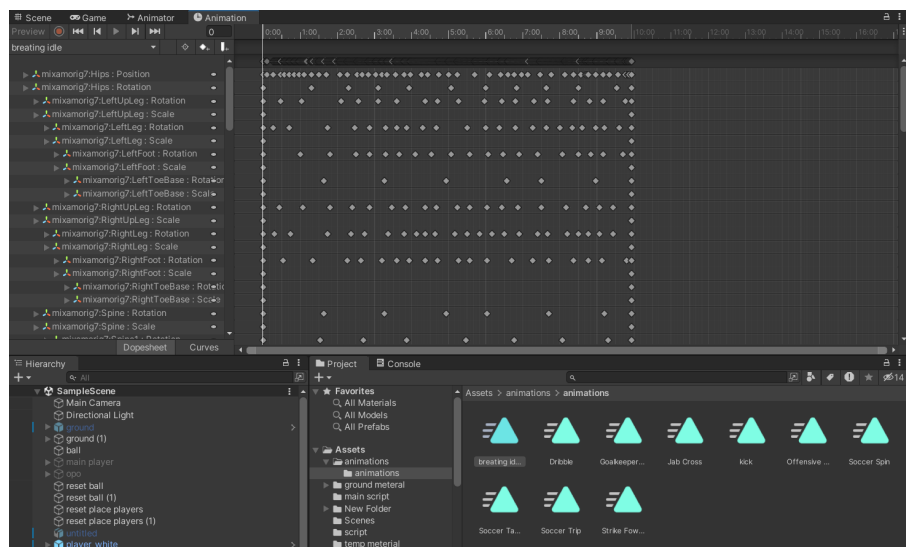


Fig. 2.2 – Animation window showing the breating idle clip (bone curves of the Mixamo rig) and the imported animation clips in Assets/animations.

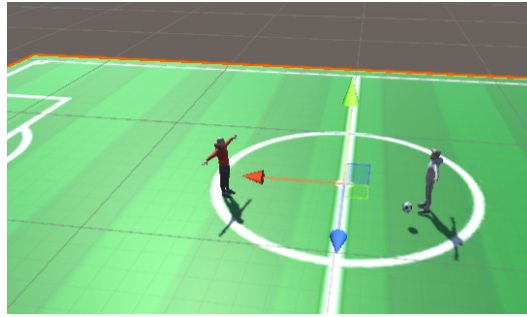


Fig. 2.3 – Play mode: the characters on the pitch with the ball.

How Unity imports and manages rigged models, animations and audio

- **Rigged models:** an FBX is processed by the Model Importer, which creates the mesh, materials and bone hierarchy. In the Rig tab the Animation Type is set to Humanoid so Unity builds an **Avatar**; this lets animations made for another Mixamo character be retargeted to mine.
- **Animations:** every clip is an AnimationClip (keyframe curves on bones, as in Fig. 2.2). Clips are arranged in an **Animator Controller** – a state machine with states, transitions and parameters (toRun, tokick, toslide, spinSkill) which my C# scripts set with `animator.SetBool()`.
- **Audio:** a sound file is imported as an AudioClip (compression, load type and mono/stereo set in the importer), played by an **AudioSource** component and heard through the **Audio Listener** on the Main Camera.

Pending item: Heightmap terrain and audio. The current game uses a flat pitch plane and has no audio files. Required for Q2: (a) GameObject → 3D Object → Terrain → Terrain Settings → Import Raw heightmap, with screenshot; (b) import a kick/whistle .wav, add an AudioSource to the player (Play On Awake off) and call `audioSource.PlayOneShot(kickClip)` inside `ballplay.cs` where `toKick` is set – this is the "audio triggered by a player action". Add screenshots of both.

Q3. Advanced Player Controller and Camera System

What is implemented

- **Movement (forward/back/left/right):** `control.cs / movement.cs` read `Input.GetAxis("Horizontal")` and `Input.GetAxis("Vertical")` and move the Rigidbody / transform; the character rotates smoothly towards the movement direction with `Quaternion.RotateTowards` / `Lerp`.
- **Animation state according to player state:** `animator.SetBool("toRun", ...)` is set true when there is movement input and false when idle (Idle ↔ Run); "spinSkill" is set when Left Shift is held while carrying the ball; "toKick" on Space with the ball; "toslide" (tackle) on Space without the ball.
- **Invalid-action protection:** in `ballplay.cs` a kick is only possible while carryingBallplayer is true, and the slide is reset by `Invoke("stopslide")` so it cannot be spammed.

Key code (control.cs)

```
float horizontalInput = Input.GetAxis("Horizontal");
float verticalInput = Input.GetAxis("Vertical");
rb.velocity = new Vector3(verticalInput, 0, -horizontalInput) * speed * Time.deltaTime;
Vector3 movement = new Vector3(verticalInput, 0, -horizontalInput).normalized;
if (movement != Vector3.zero) {
    Quaternion rotationMove = Quaternion.LookRotation(movement);
    rotationMove = Quaternion.RotateTowards(transform.rotation, rotationMove, 360 * Time.fixedDeltaTime);
    rb.MoveRotation(rotationMove);
}
animator.SetBool("toRun", horizontalInput != 0 || verticalInput != 0);
```

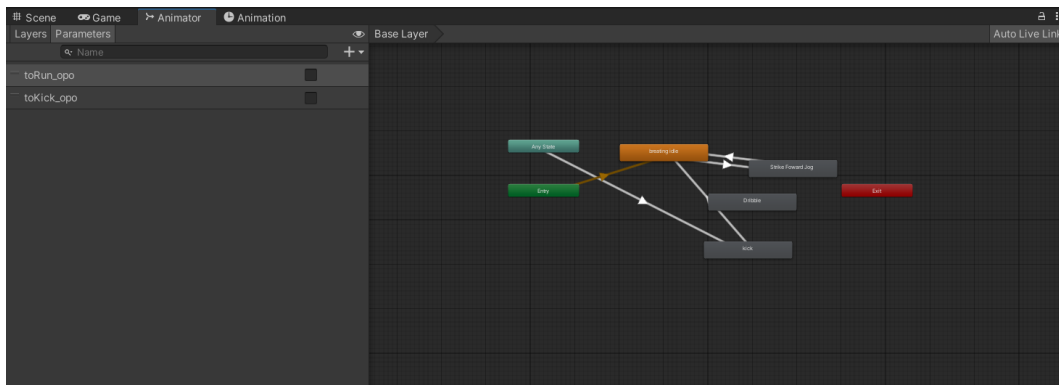


Fig. 3.1 – Animator state machine with transitions between idle, jog/run, dribble and kick (opponent controller; the player uses `player_ani` with parameters `tokick`, `toslide`, `toRun`, `spinSkill`).

Pending item: Remaining Q3 requirements. The assignment asks for Unity's **Input System** package, jump, crouch, walk/run speeds, acceleration/deceleration and a smooth-follow camera. The existing scripts use the legacy Input Manager and a fixed camera. A ready-made implementation is already in this repository: `Scripts/PlayerController.cs` and `Scripts/SmoothCameraFollow.cs` (Input System, jump guard, crouch, run, MoveTowards acceleration, SmoothDamp camera). Add them to a test scene/character, add the Input Actions asset, and capture: Input Actions editor, Animator with Idle→Walk→Run→Jump, and the Game view in each state.

Q4. Level Design, Collision Detection and Scene Management

Implemented level and collision logic

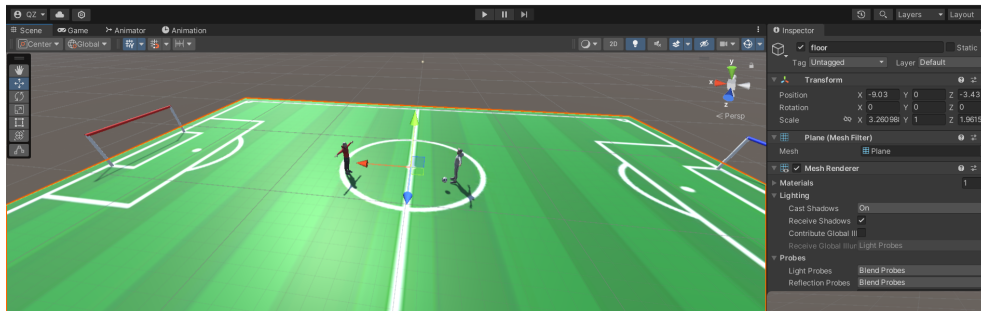


Fig. 4.1 – Level layout: pitch with start positions (centre circle), goal posts at both ends and boundaries.

Requirement	Implementation in project
Terrain / platform	Football pitch (floor/ground plane with textured material)
Colliders & Rigidbody	Players and ball have Rigidbody + colliders; ball is moved by AddForce (kick/tackle) so physics decide its movement
Triggers – goals	backtoplace.cs on opoNetgoal and playerNetgoal: OnTriggerEnter detects the ball, adds a point, zeroes velocities and teleports ball/players back to the reset points after a delay
Triggers – boundaries	boundry objects and BoundaryController.cs keep objects inside the pitch (Mathf.Clamp on X/Z)
Collision events	ballplay.OnCollisionEnter: if the player collides with the opponent while carrying the ball the ball is dropped and pushed (tackle), with a 2 s cool-down; MoveToTarget.OnCollisionEnter lets the opponent pick the ball up
Interactive object	The ball (picked up when within pickupDistance, kicked with Space)
Start / end area	Start: centre circle reset point. Goals act as event/goal areas that award score
UI	scoretext.cs updates a TextMeshPro score "Player: x Opponent: y"

Goal trigger code (backtoplace.cs, simplified)

```
void OnTriggerEnter(Collider other) {  
    if (other.gameObject == ball) {  
        ball.GetComponent<Rigidbody>().velocity = Vector3.zero;  
        ball.transform.position = ball_reset.position + new Vector3(0, 1, 0);  
        foreach (GameObject obj in GameObject.FindGameObjectsWithTag("players")) { /* reset players */ }  
        if (transform.name == "opoNetgoal") playerPoints++;  
        else if (transform.name == "playerNetgoal") opponentPoints++;  
    }  
}
```

Pending item: Scene Management and obstacles. The project has a single scene (SampleScene; Build Settings list is empty) and no dangerous obstacles. Required: (a) at least five obstacles (e.g. cones/walls with colliders, one hazard that reduces health); (b) a second scene (e.g. MainMenu or Level2) and a script using SceneManager.LoadScene – ready-made: Scripts/SceneLoader.cs, GoalTrigger.cs, DangerousObstacle.cs, PlayerHealth.cs; (c) File → Build Settings → Add Open Scenes for both scenes. Capture both scenes, the Build Settings window and Console output.

Q5. Game Performance Profiling and Optimization

Scenarios and what must be measured. Open Window → Analysis → Profiler (CPU Usage, GPU Usage, Memory, Rendering, Physics) and record each scenario for about 10 seconds. FPS can be read from the Game view Stats button (visible in Fig. 0).

Performance table - BEFORE optimization (fill with measured values)

Scenario	CPU (ms)	GPU (ms)	Memory (MB)	FPS
Main Menu				
Normal Gameplay (player + opponent + ball)				
High-Density Scene (e.g. 200+ extra balls)				
Heavy Effects (many shadowed lights/particles)				

Bottlenecks that are present in this project (found by code review - confirm with the Profiler)

#	Bottleneck	Where	Optimization
1	Excessive Debug.Log in Update - prints 8 lines every frame; Console logging is expensive and allocates strings	ballplay.cs Update()	Remove / wrap in #if UNITY_EDITOR or a debug flag
2	FindGameObjectsWithTag on every goal and GetComponent calls repeated at runtime	backtoplace.cs	Cache the player array and Rigidbodies in Awake
3	Real-time shadows and large uncompressed textures (several large pitch/audience jpgs)	Directional Light, ground metarial textures	Lower Shadow Distance, set texture Max Size 1024 + compression, disable unused textures
4	Physics in Update / unnecessary physics: rb.velocity set from Update, unused trigger callbacks	control.cs, BoundaryC ontroller.cs	Move physics to FixedUpdate, remove empty OnTrigger methods and Update clamping
5	Unused objects/scripts (empty NewBehaviourScript, trajectory, playerGoal, oponentGoal; duplicate ground and prefabs)	Assets/script, Hierarchy	Delete or disable unused GameObjects and scripts

Performance table - AFTER optimization and comparison (fill with measured values)

Scenario	FPS before	FPS after	CPU ms before → after	Memory before → after
Normal Gameplay				
High-Density Scene				
Heavy Effects				

Pending item: Profiler data. No Profiler screenshots or measurements exist yet. Run the game, capture the Profiler (CPU, GPU, Memory) for each scenario before and after applying at least three of the optimizations above, and paste the screenshots under this section. Do not copy numbers from elsewhere - the values must be your own measurements. Scripts/PerformanceLogger.cs (on-screen FPS/memory) and ObjectSpawner.cs (high-density test) are available.

Summary of Completion Status

Question	Covered by existing project	Still to add before submission
Q1	3D models, textures, materials, lighting & shadows, explanation	Shader comparison (Standard vs Unlit) and second lighting setup - screenshots
Q2	Rigged Mixamo characters, several animations, Animator Controllers, explanation	Heightmap terrain; audio clip and AudioSource trigger
Q3	Movement, rotation, animator parameters, guarded kick/slide	Input System, jump, crouch, run/acceleration, smooth camera (scripts provided in Scripts/)
Q4	Colliders, triggers, Rigidbody physics, goal logic, boundaries, interactive ball	5 obstacles/health, second scene + SceneManager script, Build Settings
Q5	Bottleneck analysis from the code	All Profiler measurements, screenshots and before/after table